KU LEUVEN

Verslag geavanceerde computerarchitectuur

Sessie 5: Dynamic parallelism and asynchronous operation

Daan Vercammen
Klaas Meersman

ELICT: ELECFS
19 maart 2025
Master in industriële wetenschappen
Academiejaar 2024-2025

1.1 Probleemstelling en doel

In dit labo onderzoeken we de invloed van het overlappen van CPU- en GPU-activiteiten via CUDA-streams. Wanneer alles sequentieel of synchroon uitgevoerd wordt, zal het potentieel van parallellisatie verloren gaan. We kunnen dus asynchroon werken in het geval er geen dependencies zijn, zo worden de CPU-dataverwerking en GPU-kernels gelijktijdig uitvoerd. We meten het performantieverschil.

1.2 Methodologie

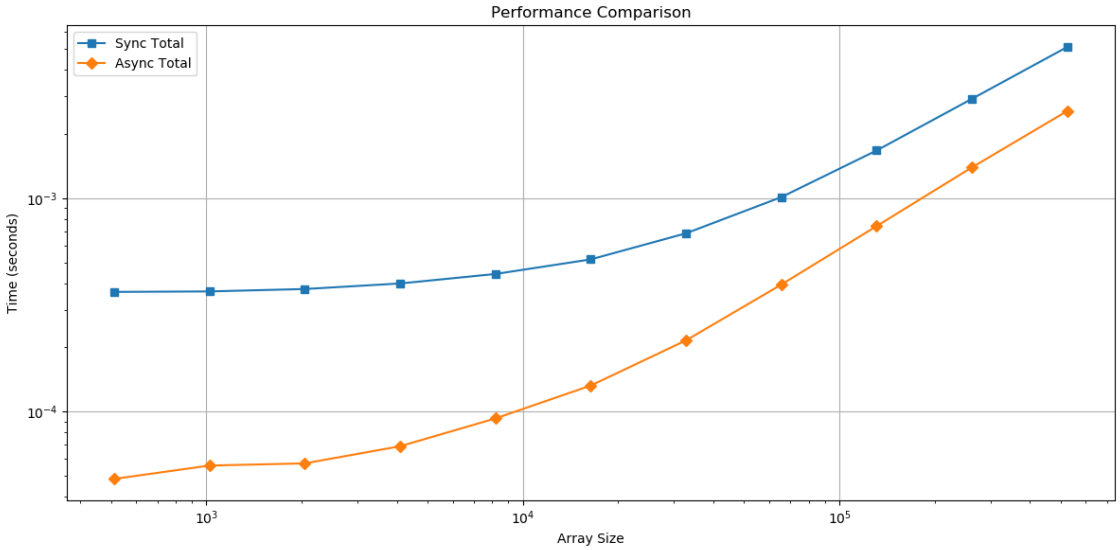
Om het verschil tussen synchrone en asynchrone uitvoering in dit labo te testen, implementeren we twee workflows: een synchrone en een asynchrone. De synchrone workflow voert alle bewerkingen sequentieel uit, waarbij elke stap pas begint nadat de vorige is voltooid. De asynchrone workflow maakt gebruik van CUDA-streams en asynchrone functies zoals cudaMemcpyAsync om CPU-dataverwerking en GPU-kerneluitvoering te overlappen, waardoor de totale uitvoeringstijd kan worden verkort. Deze test wordt uitgevoerd op vier reductieoperaties: som, product, minimum en maximum, die elk worden uitgevoerd op onafhankelijke arrays.

1.3 Resultaten en observaties

In Figuur 1.1 zien we dat het absolute verschil in uitvoeringstijd tussen synchroon (appendix A lijn 87) en asynchroon (appendix A lijn 169) werken groter wordt naarmate de arraygrootte toeneemt. Echter, het relatieve verschil (bijvoorbeeld de percentage verbetering) wordt kleiner. Dit komt doordat het aantal reductiestappen (O(log(n))) minder snel stijgt dan de hoeveelheid data die gekopieerd moet worden (O(n)). Het kopiëren heeft uiteindelijk een grotere invloed heeft op de totale uitvoeringstijd. Het effect van de reductiestappen wordt hierdoor minder dominant en het geheugen kopiëren wordt een belangrijke bottleneck.

1.4 Conclusie

De resultaten laten zien dat asynchrone uitvoering met CUDA-streams een significante prestatieverbetering biedt ten opzichte van synchrone uitvoering. Dit komt doordat asynchrone uitvoering CPU- en GPU-activiteiten overlapt, waardoor de totale uitvoeringstijd wordt geminimaliseerd. Echter, wanneer één bewerking aanzienlijk langer duurt dan de rest, zal de impact van asynchroon werken afnemen.



Figuur 1.1: Verschil synchroon vs asynchroon

A CODE

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <chrono>
#include <iostream>
#include <vector>
#include <random>
#include <ctime>
#include <algorithm>
#include <math.h>
#include <cfloat>

struct info {
    float kernelTime;
    float totalTime;
    float outcome[4];
};

__global__ void reductionMAXGPU(float *d_array, int size) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < size) ? d_array[i] : -FLT_MAX;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = max(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }
    if (tid == 0) d_array[blockIdx.x] = sdata[0];
}


__global__ void reductionMINGPU(float *d_array, int size) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < size) ? d_array[i] : FLT_MAX;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = min(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }
```

```
    if (tid == 0) d_array[blockIdx.x] = sdata[0];
}

__global__ void reductionPRODGPU(float *d_array, int size) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < size) ? d_array[i] : 1;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = sdata[tid] * sdata[tid + s];
        }
        __syncthreads();
    }
    if (tid == 0) d_array[blockIdx.x] = sdata[0];
}

__global__ void reductionSUMGPU(float *d_array, int size) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < size) ? d_array[i] : 0;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = sdata[tid] + sdata[tid + s];
        }
        __syncthreads();
    }
    if (tid == 0) d_array[blockIdx.x] = sdata[0];
}

info runGPUReductionSync(float array_SUM[], float array_PROD[], float array_MIN[], float
    array_MAX[],float size){
    float *GPU_d_array_SUM;
    float *GPU_d_array_PROD;
    float *GPU_d_array_MIN;
    float *GPU_d_array_MAX;

    cudaMalloc((void**)&GPU_d_array_SUM, size * sizeof(float));
    cudaMalloc((void**)&GPU_d_array_PROD, size * sizeof(float));
    cudaMalloc((void**)&GPU_d_array_MIN, size * sizeof(float));
    cudaMalloc((void**)&GPU_d_array_MAX, size * sizeof(float));

    int threadsPerBlock = 256;
    int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;

    int sharedMemSize = threadsPerBlock * sizeof(float);

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

```

```
    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    cudaMemcpy(GPU_d_array_SUM, array_SUM, size * sizeof(float), cudaMemcpyHostToDevice);
    cudaDeviceSynchronize();
    reductionSUMGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize>>>(GPU_d_array_SUM,
    size);
    cudaDeviceSynchronize();
    cudaMemcpy(GPU_d_array_PROD, array_PROD, size * sizeof(float), cudaMemcpyHostToDevice)
    ;
    cudaDeviceSynchronize();
    reductionPRODGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize>>>(GPU_d_array_PROD,
    size);
    cudaDeviceSynchronize();
    cudaMemcpy(GPU_d_array_MIN, array_MIN, size * sizeof(float), cudaMemcpyHostToDevice);
    cudaDeviceSynchronize();
    reductionMINGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize>>>(GPU_d_array_MIN,
    size);
    cudaDeviceSynchronize();
    cudaMemcpy(GPU_d_array_MAX, array_MAX, size * sizeof(float), cudaMemcpyHostToDevice);
    cudaDeviceSynchronize();
    reductionMAXGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize>>>(GPU_d_array_MAX,
    size);
    cudaDeviceSynchronize();


    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    cudaMemcpy(array_SUM, GPU_d_array_SUM, sizeof(float), cudaMemcpyDeviceToHost);
    cudaMemcpy(array_PROD, GPU_d_array_PROD, sizeof(float), cudaMemcpyDeviceToHost);
    cudaMemcpy(array_MIN, GPU_d_array_MIN, sizeof(float), cudaMemcpyDeviceToHost);
    cudaMemcpy(array_MAX, GPU_d_array_MAX, sizeof(float), cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    cudaFree(GPU_d_array_SUM);
    cudaFree(GPU_d_array_PROD);
    cudaFree(GPU_d_array_MIN);
    cudaFree(GPU_d_array_MAX);

    info info;
    info.kernelTime = ms_kernel / 1000.0;
    info.totalTime = ms_total / 1000.0;
    info.outcome[0] = array_SUM[0];
    info.outcome[1] =  array_PROD[0];
    info.outcome[2] = array_MIN[0];
    info.outcome[3] =  array_MAX[0];
```

```
166     return info;
167 }
168 
169 info runGPUReductionAsync(float array_SUM[], float array_PROD[], float array_MIN[], float
        array_MAX[], float size) {
170     float *GPU_d_array_SUM, *GPU_d_array_PROD, *GPU_d_array_MIN, *GPU_d_array_MAX;
171     cudaStream_t stream1;
172     cudaStream_t stream2;
173     cudaStreamCreate(&stream1);
174     cudaStreamCreate(&stream2);
175 
176     cudaMalloc((void**)&GPU_d_array_SUM, size * sizeof(float));
177     cudaMalloc((void**)&GPU_d_array_PROD, size * sizeof(float));
178     cudaMalloc((void**)&GPU_d_array_MIN, size * sizeof(float));
179     cudaMalloc((void**)&GPU_d_array_MAX, size * sizeof(float));
180 
181     int threadsPerBlock = 256;
182     int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;
183     int sharedMemSize = threadsPerBlock * sizeof(float);
184 
185     cudaEvent_t start_cuda_total, stop_cuda_total, start_cuda_kernel, stop_cuda_kernel;
186     cudaEventCreate(&start_cuda_total);
187     cudaEventCreate(&stop_cuda_total);
188     cudaEventCreate(&start_cuda_kernel);
189     cudaEventCreate(&stop_cuda_kernel);
190 
191     cudaEventRecord(start_cuda_total, stream1);
192     cudaEventRecord(start_cuda_kernel, stream1);
193 
194     // Asynchronous memory copies and kernel launches
195     cudaMemcpyAsync(GPU_d_array_SUM, array_SUM, size * sizeof(float),
        cudaMemcpyHostToDevice, stream1);
196     reductionSUMGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize, stream2>>>(
        GPU_d_array_SUM, size);
197 
198     cudaMemcpyAsync(GPU_d_array_PROD, array_PROD, size * sizeof(float),
        cudaMemcpyHostToDevice, stream1);
199     reductionPRODGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize, stream2>>>(
        GPU_d_array_PROD, size);
200 
201     cudaMemcpyAsync(GPU_d_array_MIN, array_MIN, size * sizeof(float),
        cudaMemcpyHostToDevice, stream1);
202     reductionMINGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize, stream2>>>(
        GPU_d_array_MIN, size);
203 
204     cudaMemcpyAsync(GPU_d_array_MAX, array_MAX, size * sizeof(float),
        cudaMemcpyHostToDevice, stream1);
205     reductionMAXGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize, stream2>>>(
        GPU_d_array_MAX, size);
206 
207     cudaEventRecord(stop_cuda_kernel, stream1);
208 
209     // Asynchronous copy of results back to host
210     cudaMemcpyAsync(array_SUM, GPU_d_array_SUM, sizeof(float), cudaMemcpyDeviceToHost,
        stream1);
211     cudaMemcpyAsync(array_PROD, GPU_d_array_PROD, sizeof(float), cudaMemcpyDeviceToHost,
        stream1);
212     cudaMemcpyAsync(array_MIN, GPU_d_array_MIN, sizeof(float), cudaMemcpyDeviceToHost,
        stream1);
213     cudaMemcpyAsync(array_MAX, GPU_d_array_MAX, sizeof(float), cudaMemcpyDeviceToHost,
        stream1);
```

```
214
215     cudaEventRecord(stop_cuda_total, stream1);
216     cudaStreamSynchronize(stream1);
217
218     float ms_kernel, ms_total;
219     cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);
220     cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);
221
222     cudaFree(GPU_d_array_SUM);
223     cudaFree(GPU_d_array_PROD);
224     cudaFree(GPU_d_array_MIN);
225     cudaFree(GPU_d_array_MAX);
226     cudaStreamDestroy(stream1);
227     cudaStreamDestroy(stream2);
228
229     info info;
230     info.kernelTime = ms_kernel / 1000.0;
231     info.totalTime = ms_total / 1000.0;
232     info.outcome[0] = array_SUM[0];
233     info.outcome[1] = array_PROD[0];
234     info.outcome[2] = array_MIN[0];
235     info.outcome[3] = array_MAX[0];
236     return info;
237 }
238
239 float* generateRandomArray(size_t size) {
240     float* array = new float[size];
241
242     std::random_device rd;
243     std::mt19937 gen(rd());
244     std::uniform_real_distribution<float> dis(0.0f, 1.0f);
245
246     for (size_t i = 0; i < size; ++i) {
247         do {
248             array[i] = dis(gen);
249         } while (array[i] == 0.0f || array[i] == 1.0f);
250     }
251
252     std::uniform_int_distribution<size_t> posDis(0, size - 1);
253     size_t maxPosition = posDis(gen);
254
255     array[maxPosition] = 0.999999f;
256     return array;
257 }
258
259 void printArray(int* array, size_t size) {
260     for (size_t i = 0; i < size; ++i) {
261         std::cout << array[i] << " ";
262     }
263     std::cout << std::endl;
264     printf("\n");
265 }
266
267 float getMedian(std::vector<float>& v) {
268     size_t n = v.size() / 2;
269     std::nth_element(v.begin(), v.begin()+n, v.end());
270     return v[n];
271 }
272
273 int main() {
274     const int NUM_RUNS = 20;
```

```
    const int NUM_SIZES = 11;
    const int sizes[NUM_SIZES] = {512, 1024, 2048, 4096, 8192, 16384, 32768, 65536,
    131072, 262144, 524288};

    for (int size_index = 0; size_index < NUM_SIZES; size_index++) {
        int size = sizes[size_index];
        std::vector<float> sync_kernel_times(NUM_RUNS);
        std::vector<float> sync_total_times(NUM_RUNS);
        std::vector<float> async_kernel_times(NUM_RUNS);
        std::vector<float> async_total_times(NUM_RUNS);

        for (int run = 0; run < NUM_RUNS; run++) {
            float* temp = generateRandomArray(size);
            float* my_array = new float[size];
            memcpy(my_array, temp, size * sizeof(float));
            delete[] temp;

            // Synchronous run
            float* my_array_SUM = new float[size];
            float* my_array_PROD = new float[size];
            float* my_array_MIN = new float[size];
            float* my_array_MAX = new float[size];

            memcpy(my_array_SUM, my_array, size * sizeof(float));
            memcpy(my_array_PROD, my_array, size * sizeof(float));
            memcpy(my_array_MIN, my_array, size * sizeof(float));
            memcpy(my_array_MAX, my_array, size * sizeof(float));

            info results_sync = runGPUReductionSync(my_array_SUM, my_array_PROD,
    my_array_MIN, my_array_MAX, size);
            sync_kernel_times[run] = results_sync.kernelTime;
            sync_total_times[run] = results_sync.totalTime;

            // Asynchronous run
            float* my_array_SUM2 = new float[size];
            float* my_array_PROD2 = new float[size];
            float* my_array_MIN2 = new float[size];
            float* my_array_MAX2 = new float[size];

            memcpy(my_array_SUM2, my_array, size * sizeof(float));
            memcpy(my_array_PROD2, my_array, size * sizeof(float));
            memcpy(my_array_MIN2, my_array, size * sizeof(float));
            memcpy(my_array_MAX2, my_array, size * sizeof(float));

            info results_async = runGPUReductionAsync(my_array_SUM2, my_array_PROD2,
    my_array_MIN2, my_array_MAX2, size);
            async_kernel_times[run] = results_async.kernelTime;
            async_total_times[run] = results_async.totalTime;

            delete[] my_array;
            delete[] my_array_SUM;
            delete[] my_array_PROD;
            delete[] my_array_MIN;
            delete[] my_array_MAX;
            delete[] my_array_SUM2;
            delete[] my_array_PROD2;
            delete[] my_array_MIN2;
            delete[] my_array_MAX2;
        }
        float sync_total_median = getMedian(sync_total_times);
        float async_total_median = getMedian(async_total_times);
```

```
333         printf("%d,%.6e,%.6e\n", size, sync_total_median, async_total_median);
334     }
335     return 0;
336 }
```

REFERENTIELIJST

[1] *CUDA: Determining threads per block, blocks per grid*. Stack Overflow. Geraardpleegd op 17 februari 2025. 2010. URL: https://stackoverflow.com/questions/4391162/cuda-determining-threads-per-block-blocks-per-grid.

[2] NVIDIA Corporation. *CUDA C++ Programming Guide*. Geraadpleegd op 13 februari 2025. NVIDIA. verschillend. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[3] Alex Minnaar. *Grid-stride loops*. Geraadpleegd op 3 maart 2025. Aug 2019. URL: https://alexminnaar.com/2019/08/02/grid-stride-loops.html.